Metro Ticket Generating System in ServiceNow

Capstone Project Documentation

1. Introduction

Project Title: Metro Ticketing System using ServiceNow

2. Project Overview

The Metro Ticket Generating System uses the ServiceNow platform to make buying public transit tickets digital and easy. By replacing long lines at manual ticket counters with a simple online portal, this project saves time for passengers and shows how business software can solve every day travel problems.

Purpose

To build a paperless, automated ticketing system that reduces passenger wait times and makes station management easier using digital QR codes.

Features

- Easy Booking Portal: A simple webpage where passengers can choose their starting point, destination, and number of tickets.
- Instant Fare Calculation: The system automatically calculates the exact ticket price as soon as the stations are selected, without needing a person to calculate it.
- QR Code Tickets: Once booked, the system automatically creates a unique QR code that acts as a digital ticket for passengers to scan at the entry gates.
- Creative Use of ServiceNow: Proves that ServiceNow is not just for internal company IT support, but can be used to run a real-world public transportation system.

3. Architecture

- **Frontend:** ServiceNow Service Portal, Service Catalog, and Custom UI Widgets
- **Backend:** Catalog Client Scripts, Script Includes
- **Database:** ServiceNow Relational Tables (sc_req_item, custom station/fare tables)

The Metro Ticket Generating System is built on a structured, multi-tier ServiceNow architecture. This design separates the user interface, core processing logic, and data storage to ensure a fast, scalable, and highly automated ticketing experience.

1. Presentation Layer (Frontend)

The user interface is built natively on the ServiceNow Service Portal using Service Catalog items. This provides a clean, mobile-responsive portal where commuters input their travel details (starting station, destination, journey type, and passenger count). The frontend utilizes onChange Client Scripts to capture user inputs instantly, while custom UI widgets (like spModal) are used to display the final generated QR code directly on the user's screen.

2. Application Logic Layer (Backend Processing)

The backend acts as the intelligent engine of the application, driving automation without manual intervention. It consists of:

- **GlideAjax & Script Includes:** These handle asynchronous server calls. When a user selects a route, GlideAjax securely fetches the pricing data from the server in real-time to calculate the dynamic fare before the form is even submitted.

3. Data Layer (Database)

All ticketing and infrastructure data is securely stored within ServiceNow's robust database architecture:

- **Transaction Storage:** User bookings are stored as Requested Items (RITM) in the sc_req_item table, capturing the ticket ID, travel date, fare paid, and passenger count.
- **Reference Data:** Custom tables (e.g., u_metro_stations) act as the backend database for the physical metro layout, ensuring that adding new stations or updating fare rules does not require changing the core code.

4. Integration & Output Layer

The final deliverables of the system are handled by integrated output engines. A QR code generator script encodes the commuter's unique travel data (RITM number, route, and date) into a scannable digital format. Simultaneously, the native ServiceNow Notification Engine dispatches an automated backup receipt containing the ticket details to the user's email.

5. Reporting & Analytics Layer

To support station management, the architecture utilizes ServiceNow’s native reporting tools to build real-time dashboards. These visualizations pull directly from the database layer to track peak travel times, daily ticket revenue, and the most heavily congested transit routes.

4. Setup Instructions

Prerequisites

Unlike traditional web applications requiring local server environments (e.g., Node.js, MongoDB), this solution is entirely cloud-native. The dependencies are:

- **ServiceNow Environment:** A dedicated ServiceNow Personal Developer Instance (PDI).
- **Access Credentials:** System Administrator (admin) role privileges to configure tables, scripts, and workflows.
- **Web Browser:** A modern web browser (Chrome, Edge, Firefox) for development and portal access.

Installation & Configuration Guide

Since the platform is hosted on ServiceNow, the "installation" process involves configuring the instance architecture, setting up the required data models, and establishing the automation rules.

Step 1: Database & Table Configuration

- Navigate to **System Definition > Tables**.
- Create backend reference tables for the transit infrastructure (e.g., u_metro_station_details to store station names).
- Ensure the core Requested Item (sc_req_item) table is active to store submitted ticket transactions.

Step 2: Service Catalog & Frontend Setup

- Navigate to **Service Catalog > Maintain Items** and create a new Catalog Item named "Book A Metro Ticket".
- Add the necessary variables for user input:
    - starting_from(Reference)
    - going_to (Reference)
    - type_of_journey (Single or Return options)
    - no_of_passengers(Select Box)
    - amount_for_single_journey (Single Line Text)
    - amount_including_return(Single Line Text)
    - mode_of_payment (Select Box: UPI, Card, Net Banking, Others)
    - enter_payment_mode (Single Line Text)

Step 3: UI Policies (Dynamic Form Behavior)

- Navigate to the **UI Policies** related list at the bottom of your Catalog Item.
- Create a new UI Policy to handle payment edge cases.
- **Condition:** Set the condition to [Mode of Payment] [is] [Others].
- **UI Policy Action:** Create an action for the enter_payment_method variable, setting its **Mandatory** state to True and **Visible** state to True. (This ensures the manual entry field only appears and is required when the user selects "Others").

Step 4: Client-Side Scripting (Fare Engine)

- Navigate to the **Catalog Client Scripts** related list on your item.
- Create an onChange script tied to the no_of_passengers variable.
- Insert the custom JavaScript logic to dynamically calculate amount_for_single_journey and amount_including_return based on the selected stations without requiring a page reload.

Step 5: Backend Logic & Integrations

- Navigate to **System Definition > Script Includes**.
- Create reusable, server-side functions (e.g., GlideAjax endpoints) if external API calls or complex database queries are required for the QR code generation.

Step 6: Output & Notifications Configuration

- Navigate to **System Notification > Email > Notifications**.
- Create a custom email template triggered by the Flow Designer or upon sc_req_item insertion.
- Include dynamic variables in the email body (e.g., ${number}, ${variables.starting_from}) and embed the generated QR code widget link so the commuter receives their digital pass.

Step 7: Analytics & Dashboards

- Navigate to **Reports > View / Run** and create localized reports (e.g., Bar charts for "Revenue by Route" and Time Series for "Daily Ticket Volumes").
- Navigate to **Self-Service > Dashboards** and assemble these reports into a unified Transit Admin Dashboard for real-time operational monitoring.

Step 8: End-to-End System Validation

- Impersonate a standard end-user and navigate to the Service Portal.
- Submit a test booking, ensuring the UI Policy triggers correctly, the fare calculates accurately, the backend database logs the RITM, and the email notification containing the QR code is successfully delivered to the system outbox.

5. Component & Module Structure

Because this solution is built natively on the ServiceNow platform, it does not utilize a traditional local file system repository (like a standard React/Node.js stack). Instead, the architectural "folder structure" is organized into distinct logical modules within the platform's cloud environment:

- **Frontend - Presentation & Client Logic**
    - **Service Catalog Items:** The core user-facing digital forms defining the input variables (Source, Destination, Passenger Count, Mode of Payment).
    - **UI Policies:** Configuration rules governing dynamic form behavior, such as ensuring the "Enter Payment Method" text field becomes strictly mandatory and visible only when the user selects "Others" from the payment dropdown.
    - **Catalog Client Scripts:** Frontend JavaScript executing directly in the commuter's browser (e.g., onChange scripts) to provide real-time input validation and instant dynamic fare previews without requiring page reloads.

- **Service Portal Widgets:** Custom UI components (such as spModal) responsible for rendering the final digital QR code ticket directly on the user's screen.

- **Backend - Server Side Business Logic**
  - **Script Includes:** Reusable server-side JavaScript libraries that process complex data requests safely away from the client. This includes housing the core fare price logic engine securely queried via GlideAjax to calculate accurate journey costs as well as the dedicated backend functions required for dynamic QR code generation.

- **Database - Data Management**
  - **Transactional Tables (**sc_req_item**):** The primary relational table storing individual commuter ticket records, capturing specific journey metadata, payment statuses, and generated QR references.
  - **Custom Reference Tables:** Dedicated structural tables (e.g., u_metro_station_details) storing static infrastructure data, ensuring that routing and fare updates are handled via simple database entries rather than hardcoded logic.

- **Reporting & Analytics Layer**
  - **Reports:** Configured data queries generating specific visualizations (e.g., bar charts for revenue, time-series graphs for daily bookings).
  - **Dashboards:** Centralized administrative hubs that aggregate individual reports, providing transit managers with real-time operational visibility into station traffic and financial metrics.

Because this solution is hosted entirely natively on the ServiceNow cloud platform, it does not require local terminal commands like npm start to spin up frontend or backend servers. Instead, the application is "run" continuously by the instance and is executed by accessing the live Service Portal.

6. Running the Application (Execution & Workflow)

Step 1: Accessing the Frontend (Service Portal)

- Users open the ServiceNow instance URL and navigate directly to the Service Portal
- The user searches for and opens the Book A Metro Ticket Catalog Item.

Step 2: Executing the User Journey & Client Logic

- The commuter inputs their journey details (Source, Destination, Passenger Count, Journey Type).
- **Dynamic Fare Preview:** As the user selects stations, the frontend onChange Client Scripts instantly calculate and populate the expected ticket cost on the screen.

- **UI Policy Execution:** The user selects a payment method. If they select "Others", the custom UI Policy immediately triggers, displaying the "Enter Payment Method" text field and making it mandatory before proceeding.

Step 3: Triggering the Backend (Form Submission)

- The user clicks **Submit**.
- The ServiceNow platform seamlessly transitions the data from the frontend portal to the backend database, automatically creating a new Requested Item (RITM) in the sc_req_item table.

Step 4: Server-Side Automation (Business Rules & Script Includes)

- Once the RITM is inserted into the database, the backend architecture takes over automatically.
- **Fare & Data Validation:** Business Rules execute to validate the final submission data and lock in the official fare price logic.
- **QR Code Generation:** The Business Rules and Flow Designer trigger the Script Includes, which securely generate the unique QR code linking to the commuter's specific ticket parameters.

Step 5: Output Delivery (Notifications)

- With the processing complete, the ServiceNow Notification Engine automatically dispatches an email to the commuter.
- The email contains the finalized booking details and the rendered QR Code, acting as their official digital travel pass.

Step 6: Administrative Monitoring

- Transit administrators log into the standard ServiceNow backend interface.
- They navigate to the custom Dashboards to view real-time data visualizations of the newly generated tickets, validating system health, revenue generation, and route popularity without needing to access a separate database client.

7. Scripting and Automation Logic

The Metro Ticket Generating System relies on native ServiceNow client-side scripting and Service Portal widgets to handle its core business logic. This ensures that ticket processing, dynamic fare calculation, and QR code rendering are executed seamlessly directly within the user interface.

1. Internal Logic: Dynamic Fare Engine (Catalog Client Scripts)

The frontend processes pricing natively within the user's browser using Catalog Client Scripts.

- **Trigger:** Executes instantly (onChange) whenever the user updates the source, destination, journey type, or passenger count fields.

- **Data Processed:** The script evaluates the selected starting and ending stations alongside the passenger count against predefined route conditions scripted directly into the platform.
- **Action Executed:** The client script instantly computes the pricing logic and auto-populates the exact base fare and total calculated fare directly on the user interface, providing immediate feedback without requiring a page reload.

2. Digital Pass Rendering: QR Code Generation

To generate the digital travel passes, the system integrates a custom Service Portal Widget.

- **Execution Method:** Executed via client-side JavaScript rendering within the portal framework (utilizing native widget client controllers).
- **Data Processed:** The widget captures a concatenated string of the essential ticket metadata straight from the form inputs, including the source station, destination station, date of travel, and passenger count.
- **Output Rendered:** The widget processes the data string and dynamically draws a secure, machine-readable QR code directly on the commuter's screen.

3. Automation Trigger Points

To maintain a streamlined user experience, the automation architecture is driven directly by frontend UI events. The dynamic pricing is governed solely by user input modifications, while the QR code generation is triggered by the widget immediately upon successful form validation and submission.

4. End-to-End Data Flow Summary

- The commuter inputs their journey details on the Service Portal.
- The Catalog Client Script detects the inputs and instantly calculates and displays the correct fare.
- The commuter finalizes the booking and submits the ticket request.
- The custom Service Portal Widget captures the submitted data string.
- The widget dynamically generates and renders the visual QR code directly on the user's screen.
- Simultaneously, ServiceNow natively saves the finalized ticket record (RITM) into the backend database for administrative tracking.
- The standard Notification Engine dispatches the final email receipt to the commuter.

8. Authentication and Authorization

The Metro Ticket Generating System relies on ServiceNow's robust, built-in security framework to handle all user authentication, session management, and data authorization. By utilizing the platform's native architecture, the system ensures a secure, enterprise-grade ticketing environment without the need to build custom token-based authentication from scratch.

1. Authentication Mechanism & Session Management

User identity verification is managed entirely by the ServiceNow platform. Commuters and administrators must authenticate via the platform's login portal (or an organization's Single Sign-On integration) before accessing the Service Portal. Once authenticated, ServiceNow manages the connection using secure, encrypted session cookies, maintaining the user's logged-in state as they navigate the catalog and submit ticket requests.

2. Role-Based Access Control

System authorization is governed by predefined roles to ensure users can only interact with appropriate functionalities:

- **Commuter Role (e.g.,** snc_internal **or custom** metro_user**):** Grants access to the Service Portal, allowing users to select routes, calculate fares, submit bookings, and view their personal digital tickets.
- **Station Manager Role (**metro_manager**):** Allows transit staff to access backend modules to view all generated ticket records, monitor daily revenue dashboards, and oversee station traffic.
- **System Administrator (**admin**):** Retains complete architectural control over the application, including the ability to modify client scripts, update the custom UI widget, and manage the database tables.

3. Data Security & Access Control Lists

To prevent unauthorized data exposure or manipulation, ServiceNow Access Control Lists (ACLs) are strictly enforced at the database level:

- **Record-Level Security:** Commuters are restricted to viewing only the specific Requested Items (RITMs) they have personally booked. They cannot query or view other passengers' travel data.
- **Field-Level Security:** Administrative data, such as internal station system IDs or backend processing states, are hidden from standard users on the portal view.
- **Data Policies:** While the Catalog Client Scripts calculate the fare dynamically on the frontend, native data policies ensure that submitted records adhere to strict formatting rules before writing to the database.

4. Portal and Digital Pass Security

- **Restricted Portal Access:** The Metro Ticket Service Catalog is locked behind active authentication. Unauthenticated users attempting to access the booking URL are automatically redirected to the login screen.
- **QR Code Integrity:** Because the digital pass is rendered via a secure Service Portal widget immediately upon a validated form submission, commuters cannot arbitrarily alter the encoded routing, date, or passenger count data after the ticket has been finalized.

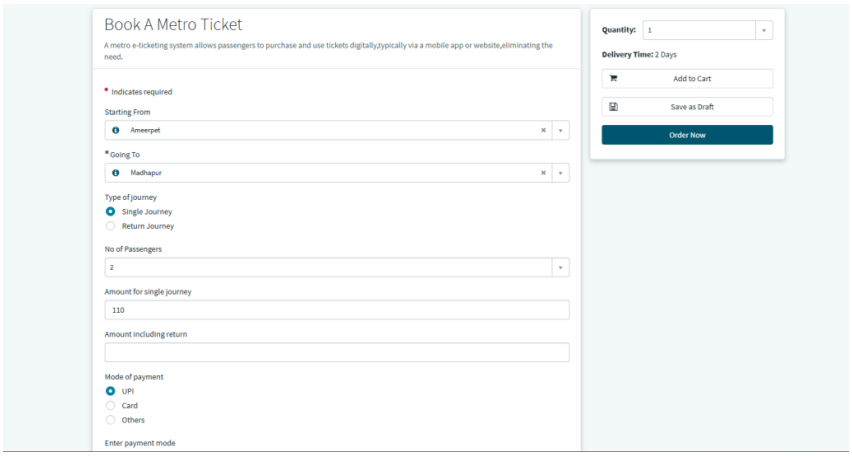
9. User Interface

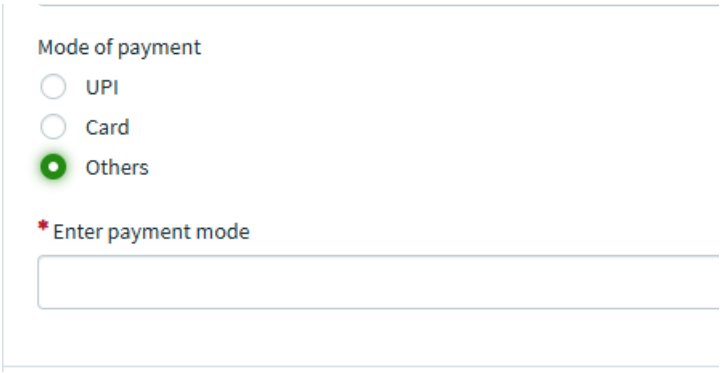
The Metro Ticket Generating System features a clean, intuitive, and mobile-friendly interface built entirely on the ServiceNow Service Portal. It is designed to make the ticket booking process as fast and simple as possible for daily commuters while providing powerful tracking tools for transit administrators.

1. Ticket Booking Form (Service Portal)

The primary commuter interface is a structured Service Catalog form.

- **Form Layout:** Features easy-to-use dropdowns and input fields for Source Station, Destination Station, Journey Type, Passenger Count, and Payment Method.
- **Dynamic UI Policies:** The form adapts to user behavior in real-time. For example, selecting "Others" in the payment dropdown immediately triggers a UI Policy that displays and mandates the "Enter Payment Method" text field.
- **Instant Fare Preview:** Driven by Catalog Client Scripts, the form auto-populates the exact ticket cost instantly as the user selects their route, preventing the need to refresh the page or manually calculate totals.

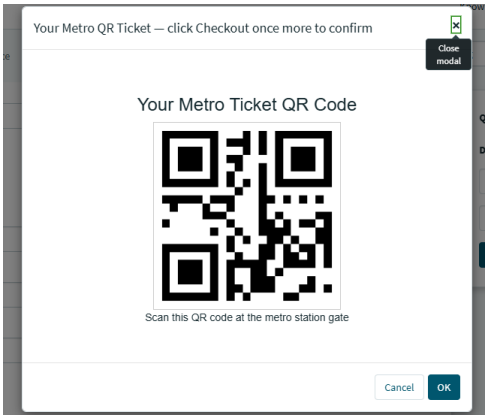




2. Digital Ticket & QR Code Rendering

Upon successful form submission, the user is presented with their finalized travel details and digital pass.

- **Custom UI Widget:** A dedicated Service Portal Widget securely captures the submitted data and renders a scannable QR code directly on the confirmation screen.
- **Ticket Details:** The screen clearly displays the unique Request Number (RITM), valid travel date, passenger count, and total paid fare.



3. Transit Administrator Dashboard

A backend analytics interface built using ServiceNow's native reporting engine provides real-time operational visibility for transit staff.

- **Visual Reports:** Features bar charts, pie charts, and time-series graphs to track peak travel times and route-wise usage distribution.
- **Financial Tracking:** Monitors total daily revenue and ticket volumes at a glance.

4. Core User Experience (UX) Principles

- **Simplicity:** The entire booking process is condensed into a single, straightforward digital form.
- **Real-Time Feedback:** Client scripts provide immediate validation and pricing updates, reducing user errors and confusion.
- **Responsiveness:** The Service Portal framework ensures the interface is highly compatible and visually appealing on both desktop browsers and mobile devices.

10. Testing

The Metro Ticket Generating System underwent rigorous manual and functional testing to validate its end-to-end performance. The testing strategy focused on ensuring the reliability of the client-side automation, the accuracy of the dynamic UI behaviors, and the successful creation of backend records within the ServiceNow platform.

1. Test Scope

Testing was conducted across the following critical application components:

- Service Catalog form interactions and data validations.
- UI Policy executions (specifically conditional field behaviors).
- Catalog Client Script performance (instant fare calculations).

- Service Portal Widget rendering (QR code generation).
- Backend database record creation (RITM insertion).
- Outbound email notification delivery.

2. Test Scenarios and Results

Test Case 1: UI Policy Execution (Payment Method)

- **Input:** The user selects "Others" from the Mode of Payment dropdown.
- **Expected Output:** The UI Policy should immediately trigger, making the Enter Payment Method text field visible and strictly mandatory before the form can be submitted.
- **Result:** Pass

Test Case 2: Dynamic Fare Calculation (Client Script)

- **Input:** The user selects Ameerpet (Source), Madhapur (Destination), Return Journey (Type), and "2" (Passenger Count).
- **Expected Output:** The onChange Catalog Client Script should instantly calculate the math and auto-populate the Amount Including Return field, while clearing out the single journey field, all without refreshing the page.
- **Result:** Pass

Test Case 3: Ticket Booking Submission

- **Input:** All mandatory fields are correctly filled, and the user clicks "Submit".
- **Expected Output:** The system should accept the data, transition the user to the order summary, and successfully generate a Request Item (RITM) record in the backend database containing the exact input values.
- **Result:** Pass

Test Case 4: Client-Side QR Code Rendering (Widget)

- **Input:** Successful submission of the ticket form.
- **Expected Output:** The custom Service Portal widget should capture the submitted form data and dynamically draw a secure, scannable QR code directly on the confirmation screen without relying on any external APIs.
- **Result:** Pass

Test Case 5: Notification Delivery

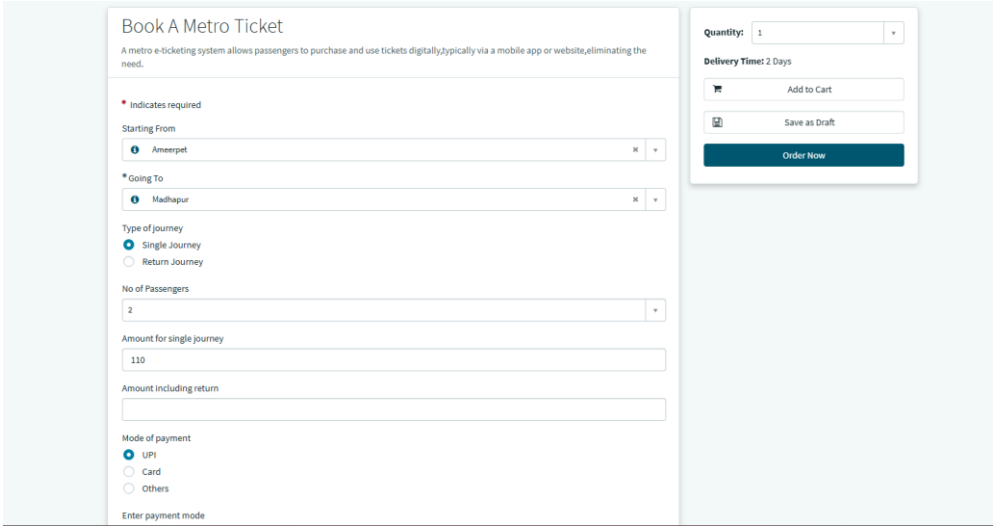
- **Input:** The backend RITM record is successfully saved.
- **Expected Output:** The native ServiceNow Notification Engine should trigger automatically, sending a confirmation email to the commuter that includes their ticket metadata and a link to their digital QR pass.

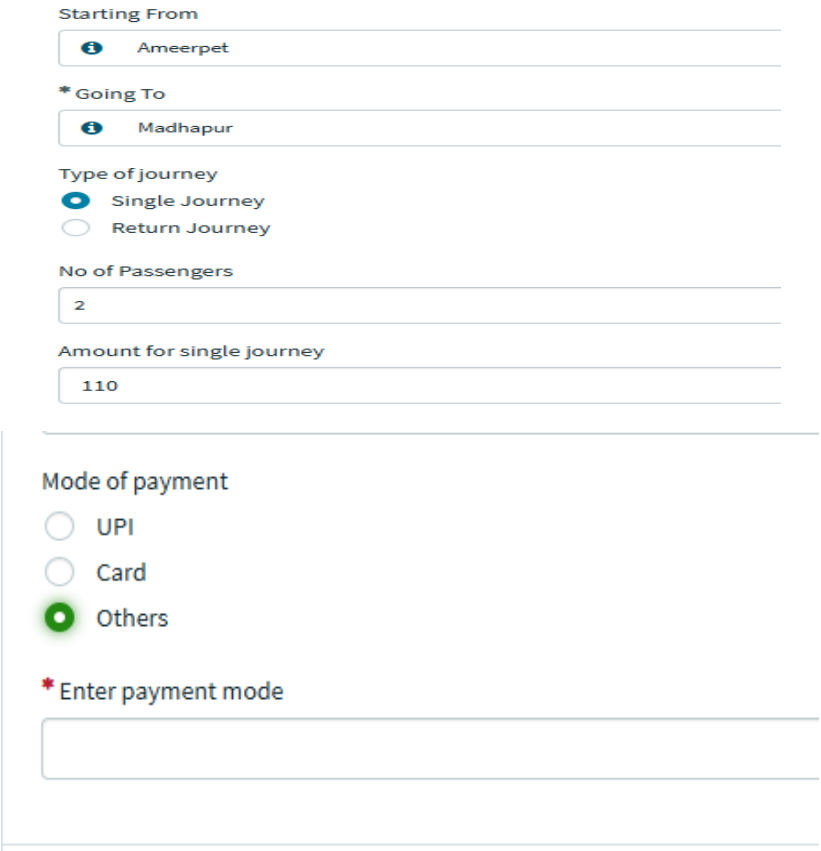
- **Result:** Pass

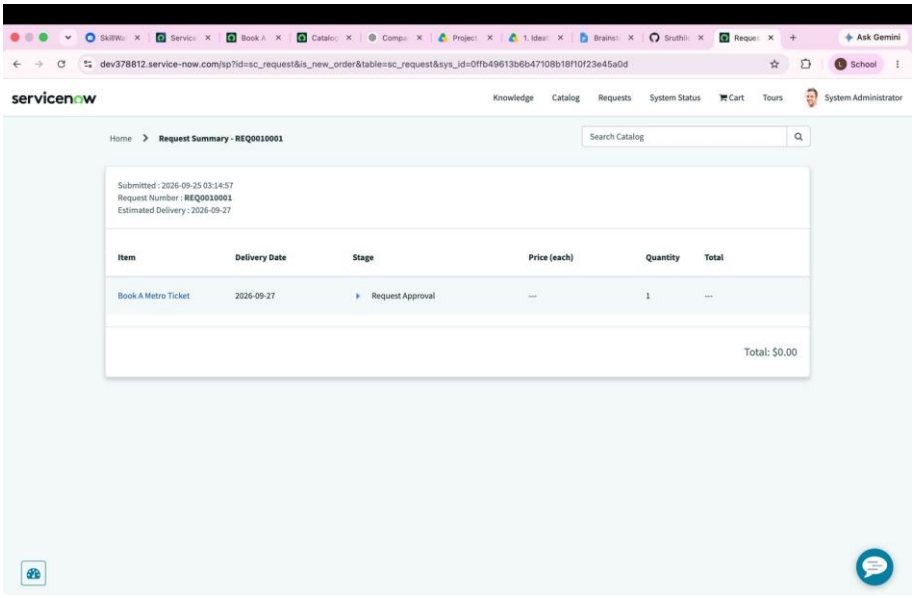
Test Case 6: Administrative Dashboard Updates

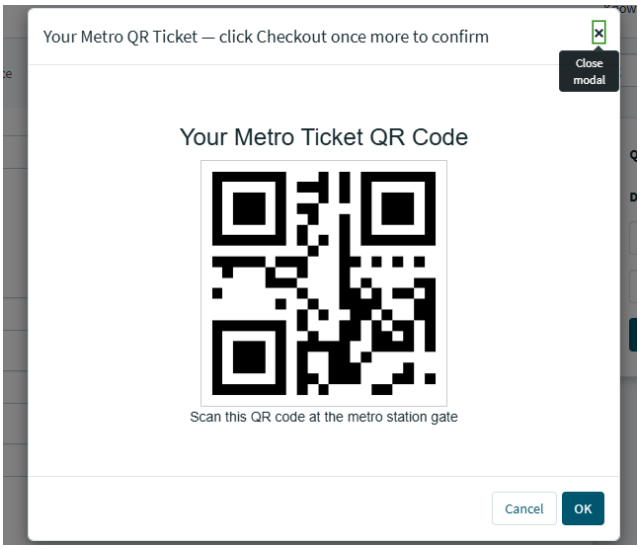
- **Input:** A new ticket is processed by the system.
- **Expected Output:** The backend transit dashboard should immediately reflect the new ticket in its volume and revenue reports, providing real-time data to station managers.
- **Result:** Pass

11. Screenshots









12. Known Issues

While the Metro Ticket Generating System successfully automates the ticketing process using native ServiceNow components, there are a few technical limitations and known issues within the current architecture.

1. Hardcoded Station Identifiers

The current dynamic fare calculation relies on an onChange Catalog Client Script that uses hardcoded station sys_id values (e.g., Ameerpet, Madhapur). If a station record is accidentally deleted and recreated in the database, its sys_id will change, which will break the fare calculation logic until the script is manually updated.

2. Client-Side Processing Reliance

Because the QR code generation is handled entirely by a Service Portal widget executing client-side JavaScript (rather than an external backend API), it relies heavily on the user's device. If a commuter accesses the portal using a very old mobile browser or has JavaScript disabled, the QR code may fail to render on their screen.

3. No Real-Time Payment Integration

While the form includes a dynamic payment selection field (triggering a mandatory UI Policy when "Others" is selected), the platform is not currently integrated with a live payment gateway like Stripe or UPI. The system successfully logs the *intended* payment method, but assumes the financial transaction was completed successfully elsewhere.

4. Static Fare Structure

The pricing logic scripted into the platform is static based on route and passenger count. The system does not currently support dynamic or surge pricing based on real-time transit congestion, peak travel hours, or holiday rates.

5. Limited Offline Validation

The generated digital QR passes require active internet connectivity to validate against the ServiceNow backend database. There is currently no offline verification system built for the physical metro entry gates if the station experiences a network outage.

6. Basic Fraud Protection

The dynamically generated QR code securely encodes the user's ticket metadata (source, destination, passenger count, RITM number). However, it does not currently utilize advanced cryptographic hashing or blockchain-level security, making it theoretically possible for a user to screenshot and share a single ticket with multiple people if gate scanners are not configured to flag duplicate entries.

13. Future Enhancements

The Metro Ticket Generating System provides a strong foundation for automated public transit ticketing using ServiceNow. To evolve this prototype into an enterprise-ready, real-world application, several enhancements can be implemented in future iterations.

1. Live Payment Gateway Integration

Currently, the system captures the user's preferred payment method via UI Policies but does not process live transactions. A major future enhancement would be integrating ServiceNow with a secure payment gateway (like Razorpay, Stripe, or UPI) using outbound REST API messages. This would allow commuters to complete their financial transactions directly on the Service Portal before the final QR code is rendered.

2. Database-Driven Fare Engine (GlideAjax)

To resolve the known issue of hardcoded station sys_ids in the Client Scripts, the pricing logic should be migrated to a backend database table (e.g., a custom u_fare_matrix table). The frontend script would be upgraded to use a GlideAjax call, dynamically fetching the fare from the database. This would allow transit administrators to add new stations or change prices easily without ever having to edit the JavaScript code.

3. Dynamic and Peak-Hour Pricing

Public transportation often relies on dynamic pricing to manage crowd control. Future updates could introduce time-based logic to the fare calculation engine, automatically applying peak-

hour surcharges during busy morning commutes or offering discounted fares on weekends and holidays.

4. Dedicated Mobile Scanner App & Cryptography

To improve fraud protection and address offline validation, a custom application could be built using the ServiceNow Mobile Agent. Transit staff at the gates could use this app on their smartphones to scan the commuter's QR codes. Additionally, the Service Portal widget could be upgraded to generate cryptographically hashed QR codes (like JWT tokens) to prevent users from forging or duplicating digital passes.

5. Multilingual Portal Localization

Because public transit serves a diverse demographic, the Service Portal should be accessible to everyone. Utilizing ServiceNow's native localization plugins, the frontend could be translated into regional languages (e.g., Telugu, Hindi) allowing users to toggle their preferred language seamlessly on the booking form.